

NAME DHRUV PANDEY**SAPID** 590037660**BATCH 15**

1.Test-Case Table

The test cases cover **valid inputs, invalid inputs, boundary values, all grade mappings, and different numbers of courses/semesters.**

Test Case	Input / Condition	Expected Output	Actual Output	R
1 – Valid, 2	ID=1001, m=2. Sem 1:	Sem 1 SGPA=9.10, Sem	SGPA=9.10,	P
2 –	ID=1, m=1, n=1,	SGPA=10.00,	SGPA=10.00,	P
3 – Lowest	ID=101, m=1, n=1,	SGPA=4.00, CGPA=4.00	SGPA=4.00,	P
4 – Invalid	ID=0. then 1001	Reiect 0 and request ID	Error displaved:	P
5 – Invalid	m=-2, then 1	Reject -2 and request m	Error displayed;	P
6 – Invalid	n=0. then 1	Reiect 0 and request n	Error displaved:	P
7 – Invalid	code=-101. then 101	Reiect -101 and request	Error displaved:	P
8 – Invalid	credit=0. then 4	Reiect 0 and request	Error displaved:	P
9 – Invalid	grade=X. then B	Reiect X and request	Error displaved:	P
10 – All	O.A.B.C.D.E.P with	Corresponding GP:	SGPA=7.00	P

Note: The algorithm only specifies that integer inputs must be **positive**; therefore, 1 is the lower boundary for `global_id`, `m`, `n`, `code`, and `credit`.

2. C Program

```
#include <stdio.h>
#include <ctype.h>
```

```

{
    int global_id, m, n;
    int semester, course;
    int code, credit, GP;
    int semester_TC, semester_TGC;
    int cumulative_TC = 0, cumulative_TGC = 0;
    char grade;
    float SGPA, CGPA;
    /* Input and validation of Global ID */
    do
    {
        printf("Enter Global ID: ");
        scanf("%d", &global_id);
        if (global_id <= 0)
            printf("Invalid Global ID. Enter a positive value.
\n");
    } while (global_id <= 0);
    /* Input and validation of number of semesters */
    do
    {
        printf("Enter number of semesters: ");
        scanf("%d", &m);
        if (m <= 0)
            printf("Invalid number of semesters. Enter a
positive value.\n");
    } while (m <= 0);
    /* Process each semester */
    for (semester = 1; semester <= m; semester++)
    {
        semester_TC = 0;
        semester_TGC = 0;
        /* Input and validation of courses */
        do
        {
            printf("\nEnter number of courses for Semester %d:
",
                    semester);

```

DHRUV PANDEY 590037660 BATCH 15

```
        sc
        &n);
    if (n <= 0)
        printf("Invalid number of courses. "
               "Enter a positive value.\n");
} while (n <= 0);
/* Process each course */
for (course = 1; course <= n; course++)
{
    printf("\nCourse %d\n", course);
    /* Course code validation */
    do
    {
        printf("Enter course code: ");
        scanf("%d", &code);
        if (code <= 0)
            printf("Invalid course code. "
                   "Enter a positive value.\n");
    } while (code <= 0);
    /* Credit validation */
    do
    {
        printf("Enter credit: ");
        scanf("%d", &credit);
        if (credit <= 0)
            printf("Invalid credit. "
                   "Enter a positive value.\n");
    } while (credit <= 0);
    /* Grade validation */
    do
    {
        printf("Enter grade (O/A/B/C/D/E/P): ");
        scanf(" %c", &grade);
        grade = toupper(grade);
        if (grade != 'O' && grade != 'A' &&
            grade != 'B' && grade != 'C' &&
            grade != 'D' && grade != 'E' &&
            grade != 'P')
        {
```

```
        ntf("Invalid grade. Enter O, A, B, "
            "C, D, E, or P.\n");
    }
} while (grade != 'O' && grade != 'A' &&
        grade != 'B' && grade != 'C' &&
        grade != 'D' && grade != 'E' &&
        grade != 'P');
/* Convert grade to Grade Point */
switch (grade)
{
    case 'O':
        GP = 10;
        break;
    case 'A':
        GP = 9;
        break;
    case 'B':
        GP = 8;
        break;
    case 'C':
        GP = 7;
        break;
    case 'D':
        GP = 6;
        break;
    case 'E':
        GP = 5;
        break;
    case 'P':
        GP = 4;
        break;
}
/* Calculate semester totals */
semester_TGC = semester_TGC + (GP * credit);
semester_TC = semester_TC + credit;
}
/* Calculate SGPA */
SGPA = (float) semester_TGC / semester_TC;
```

```

DHRUV PANDEY    590037660    BATCH 15
    tf          ter %d SGPA = %.2f\n",
        semester, SGPA);
/* Update cumulative values */
cumulative_TGC = cumulative_TGC + semester_TGC;
cumulative_TC = cumulative_TC + semester_TC;
}
/* Calculate CGPA */
CGPA = (float) cumulative_TGC / cumulative_TC;
/* Final output */
printf("\n----- FINAL RESULT ----- \n");
printf("Global ID          = %d\n", global_id);
printf("Number of Semesters   = %d\n", m);
printf("Cumulative Credits     = %d\n", cumulative_TC);
printf("CGPA                  = %.2f\n", CGPA);
return 0;
}

```

3. Compilation

I compiled the program using GCC with warnings enabled:

```
gcc -Wall -Wextra -std=c11 sgpa.c -o sgpa
```

Compilation result: Successful — no compilation errors.

Test-Case Table

The test cases cover **valid inputs, invalid inputs, boundary values, all grade mappings, and different numbers of courses/semesters.**

Test Case	Input / Condition	Expected Output	Actual Output	R
1 - Valid, 2	ID=1001, m=2. Sem 1: 9.10, 9.10, 9.10, 9.10	Sem 1 SGPA=9.10, Sem 2 SGPA=9.10	SGPA=9.10, 7.50	P
2 -	ID=1, m=1, n=1, 10.00, 10.00, 10.00	SGPA=10.00, CGPA=10.00	SGPA=10.00, CGPA=10.00	P
3 - Lowest	ID=101, m=1, n=1, 4.00, 4.00, 4.00	SGPA=4.00, CGPA=4.00	SGPA=4.00, CGPA=4.00	P
4 - Invalid	ID=0. then 1001	Reiect 0 and request ID	Error displaved:	P
5 - Invalid	m=-2, then 1	Reject -2 and request m	Error displayed;	P

6 – Invalid	n=0. then 1	Reiect 0 and request n	Error displaved:	P
7 – Invalid	code=-101. then 101	Reiect -101 and request	Error displaved:	P
8 – Invalid	credit=0. then 4	Reiect 0 and request	Error displaved:	P
9 – Invalid	grade=X. then B	Reiect X and request	Error displaved:	P
10 – All	O.A.B.C.D.F.P with	Corresponnding GP:	SGPA=7.00	P

Note: The algorithm only specifies that integer inputs must be **positive**; therefore, 1 is the lower boundary for global_id, m, n, code, and credit.

2. C Program

```
#include <stdio.h>
#include <ctype.h>
int main()
{
    int global_id, m, n;
    int semester, course;
    int code, credit, GP;
    int semester_TC, semester_TGC;
    int cumulative_TC = 0, cumulative_TGC = 0;
    char grade;
    float SGPA, CGPA;
    /* Input and validation of Global ID */
    do
    {
        printf("Enter Global ID: ");
        scanf("%d", &global_id);
        if (global_id <= 0)
            printf("Invalid Global ID. Enter a positive value.
\n");
    } while (global_id <= 0);
    /* Input and validation of number of semesters */
    do
    {
        printf("Enter number of semesters: ");
        scanf("%d", &m);
        if (m <= 0)
```

DHRUV PANDEY 590037660 BATCH 15

```
pr                    alid number of semesters. Enter a
positive value.\n");
} while (m <= 0);
/* Process each semester */
for (semester = 1; semester <= m; semester++)
{
    semester_TC = 0;
    semester_TGC = 0;
    /* Input and validation of courses */
    do
    {
        printf("\nEnter number of courses for Semester %d:
",
               semester);
        scanf("%d", &n);
        if (n <= 0)
            printf("Invalid number of courses. "
                  "Enter a positive value.\n");
    } while (n <= 0);
    /* Process each course */
    for (course = 1; course <= n; course++)
    {
        printf("\nCourse %d\n", course);
        /* Course code validation */
        do
        {
            printf("Enter course code: ");
            scanf("%d", &code);
            if (code <= 0)
                printf("Invalid course code. "
                      "Enter a positive value.\n");
        } while (code <= 0);
        /* Credit validation */
        do
        {
            printf("Enter credit: ");
            scanf("%d", &credit);
            if (credit <= 0)
```

```
        printf("Invalid credit. "
               "Enter a positive value.\n");
    } while (credit <= 0);
    /* Grade validation */
    do
    {
        printf("Enter grade (O/A/B/C/D/E/P): ");
        scanf(" %c", &grade);
        grade = toupper(grade);
        if (grade != 'O' && grade != 'A' &&
            grade != 'B' && grade != 'C' &&
            grade != 'D' && grade != 'E' &&
            grade != 'P')
        {
            printf("Invalid grade. Enter O, A, B, "
                   "C, D, E, or P.\n");
        }
    } while (grade != 'O' && grade != 'A' &&
             grade != 'B' && grade != 'C' &&
             grade != 'D' && grade != 'E' &&
             grade != 'P');
    /* Convert grade to Grade Point */
    switch (grade)
    {
        case 'O':
            GP = 10;
            break;
        case 'A':
            GP = 9;
            break;
        case 'B':
            GP = 8;
            break;
        case 'C':
            GP = 7;
            break;
        case 'D':
            GP = 6;
```

```
        ak;
        case 'E':
            GP = 5;
            break;
        case 'P':
            GP = 4;
            break;
    }
    /* Calculate semester totals */
    semester_TGC = semester_TGC + (GP * credit);
    semester_TC = semester_TC + credit;
}
/* Calculate SGPA */
SGPA = (float) semester_TGC / semester_TC;
printf("\nSemester %d SGPA = %.2f\n",
        semester, SGPA);
/* Update cumulative values */
cumulative_TGC = cumulative_TGC + semester_TGC;
cumulative_TC = cumulative_TC + semester_TC;
}
/* Calculate CGPA */
CGPA = (float) cumulative_TGC / cumulative_TC;
/* Final output */
printf("\n----- FINAL RESULT -----\n");
printf("Global ID                = %d\n", global_id);
printf("Number of Semesters      = %d\n", m);
printf("Cumulative Credits        = %d\n", cumulative_TC);
printf("CGPA                      = %.2f\n", CGPA);
return 0;
}
```

3. Compilation

I compiled the program using GCC with warnings enabled:

```
gcc -Wall -Wextra -std=c11 sgpa.c -o sgpa
```

Compilation result: Successful — no compilation errors.

Test-Case Table

The test cases cover **valid inputs, invalid inputs, boundary values, all grade mappings, and different numbers of courses/semesters.**

Test Case	Input / Condition	Expected Output	Actual Output	R
1 – Valid, 2	ID=1001, m=2. Sem 1: A, B, C, D, E	Sem 1 SGPA=9.10, Sem 2 SGPA=7.50	SGPA=9.10, 7.50	P
2 –	ID=1, m=1, n=1, A, B, C, D, E	SGPA=10.00, CGPA=10.00	SGPA=10.00, CGPA=10.00	P
3 – Lowest	ID=101, m=1, n=1, A, B, C, D, E	SGPA=4.00, CGPA=4.00	SGPA=4.00, CGPA=4.00	P
4 – Invalid	ID=0. then 1001	Reject 0 and request ID	Error displayed:	P
5 – Invalid	m=-2, then 1	Reject -2 and request m	Error displayed;	P
6 – Invalid	n=0. then 1	Reject 0 and request n	Error displayed:	P
7 – Invalid	code=-101. then 101	Reject -101 and request	Error displayed:	P
8 – Invalid	credit=0. then 4	Reject 0 and request	Error displayed:	P
9 – Invalid	grade=X. then B	Reject X and request	Error displayed:	P
10 – All	O.A.B.C.D.E.P with	Corresponding GP:	SGPA=7.00	P

Note: The algorithm only specifies that integer inputs must be **positive**; therefore, 1 is the lower boundary for global_id, m, n, code, and credit.

2. C Program

```
#include <stdio.h>
#include <ctype.h>
int main()
{
    int global_id, m, n;
    int semester, course;
    int code, credit, GP;
    int semester_TC, semester_TGC;
    int cumulative_TC = 0, cumulative_TGC = 0;
    char grade;
    float SGPA, CGPA;
    /* Input and validation of Global ID */
    do
    {
```

DHRUV PANDEY 590037660 BATCH 15

```
        tf                      llobal ID: ");
        scanf("%d", &global_id);
        if (global_id <= 0)
            printf("Invalid Global ID. Enter a positive value.
\n");
    } while (global_id <= 0);
    /* Input and validation of number of semesters */
    do
    {
        printf("Enter number of semesters: ");
        scanf("%d", &m);
        if (m <= 0)
            printf("Invalid number of semesters. Enter a
positive value.\n");
    } while (m <= 0);
    /* Process each semester */
    for (semester = 1; semester <= m; semester++)
    {
        semester_TC = 0;
        semester_TGC = 0;
        /* Input and validation of courses */
        do
        {
            printf("\nEnter number of courses for Semester %d:
",
                semester);
            scanf("%d", &n);
            if (n <= 0)
                printf("Invalid number of courses. "
                    "Enter a positive value.\n");
        } while (n <= 0);
        /* Process each course */
        for (course = 1; course <= n; course++)
        {
            printf("\nCourse %d\n", course);
            /* Course code validation */
            do
            {
```

```
        "Enter course code: ");
    scanf("%d", &code);
    if (code <= 0)
        printf("Invalid course code. "
               "Enter a positive value.\n");
} while (code <= 0);
/* Credit validation */
do
{
    printf("Enter credit: ");
    scanf("%d", &credit);
    if (credit <= 0)
        printf("Invalid credit. "
               "Enter a positive value.\n");
} while (credit <= 0);
/* Grade validation */
do
{
    printf("Enter grade (O/A/B/C/D/E/P): ");
    scanf(" %c", &grade);
    grade = toupper(grade);
    if (grade != 'O' && grade != 'A' &&
        grade != 'B' && grade != 'C' &&
        grade != 'D' && grade != 'E' &&
        grade != 'P')
    {
        printf("Invalid grade. Enter O, A, B, "
               "C, D, E, or P.\n");
    }
} while (grade != 'O' && grade != 'A' &&
        grade != 'B' && grade != 'C' &&
        grade != 'D' && grade != 'E' &&
        grade != 'P');
/* Convert grade to Grade Point */
switch (grade)
{
    case 'O':
        GP = 10;
```

DHRUV PANDEY 590037660 BATCH 15

```
        ak;
        case 'A':
            GP = 9;
            break;
        case 'B':
            GP = 8;
            break;
        case 'C':
            GP = 7;
            break;
        case 'D':
            GP = 6;
            break;
        case 'E':
            GP = 5;
            break;
        case 'P':
            GP = 4;
            break;
    }
    /* Calculate semester totals */
    semester_TGC = semester_TGC + (GP * credit);
    semester_TC = semester_TC + credit;
}
/* Calculate SGPA */
SGPA = (float) semester_TGC / semester_TC;
printf("\nSemester %d SGPA = %.2f\n",
        semester, SGPA);
/* Update cumulative values */
cumulative_TGC = cumulative_TGC + semester_TGC;
cumulative_TC = cumulative_TC + semester_TC;
}
/* Calculate CGPA */
CGPA = (float) cumulative_TGC / cumulative_TC;
/* Final output */
printf("\n----- FINAL RESULT ----- \n");
printf("Global ID                      = %d\n", global_id);
printf("Number of Semesters           = %d\n", m);
```

```
DHRUV PANDEY    590037660    BATCH 15
                Cu            Credits    = %d\n", cumulative_TC);
printf("CGPA            = %.2f\n", CGPA);
return 0;
}
```

3. Compilation

I compiled the program using GCC with warnings enabled:

```
gcc -Wall -Wextra -std=c11 sgpa.c -o sgpa
```

Compilation result: Successful — no compilation errors.

4.Execution

Execution for test 1

```
Last login: Sat Sep 12 14:34:17 on ttys000
dhruv@Dhruvs-MacBook-Air ~ % nano day19.cp
dhruv@Dhruvs-MacBook-Air ~ % clang day19.cp -o day19
dhruv@Dhruvs-MacBook-Air ~ % ./day19
Enter Global ID: 590037660
Enter number of semesters: 1

Enter number of courses for Semester 1: 3

Course 1
Enter course code: 101
Enter credit: 4
Enter grade (O/A/B/C/D/E/P): O

Course 2
Enter course code: 102
Enter credit: 3
Enter grade (O/A/B/C/D/E/P): A

Course 3
Enter course code: 103
Enter credit: 3
Enter grade (O/A/B/C/D/E/P): B

Semester 1 SGPA = 9.10

----- FINAL RESULT -----
Global ID           = 590037660
Number of Semesters = 1
Cumulative Credits  = 10
CGPA                = 9.10
dhruv@Dhruvs-MacBook-Air ~ %
```

Calculation Verification

For Semester 1:

$$TGC=(10\times 4)+(9\times 3)+(8\times 3)=91$$

$$TC=4+3+3=10$$

$$SGPA=1091=9.10$$